Python Chapter 2

Notes

Operators:

Operators are used to perform operations on variables and values.

Python divides the operators in the following groups:

- Arithmetic operators
- Assignment operators (not useful for MSBTE)
- Relational operators
- Logical operators
- Identity operators
- Membership operators (will discuss in list and tuples topic)
- Bitwise operators (not for MSBTE)

Arithmetic Operators:

Operator	Name	Example
+	Addition	x + y
-	Subtraction	x - y
*	Multiplication	x * y
/	Division	x / y
%	Modulus	x % y
**	Exponentiation	x ** y
//	Floor division	x // y

Relational Operators:

Operator	Name	Example
==	Equal	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

Logical Operators:

Operator	Description	Example
and	Returns True if both statements are true	x < 5 and x < 10
or	Returns True if one of the statements is true	x < 5 or x < 4
not	Reverse the result, returns False if the result is true	not(x < 5 and x < 10)

Identity Operators:

Operator	Description	Example
is	Returns True if both variables are the same object	x is y
is not	Returns True if both variables are not the same object	x is not y

Program For Arithmetic Operation

```
# Store input numbers:
num1 = input('Enter first number: ')
num2 = input('Enter second number: ')

# Add two numbers
sum = float(num1) + float(num2)
# Subtract two numbers
min = float(num1) - float(num2)
# Multiply two numbers
```

```
mul = float(num1) * float(num2)
#Divide two numbers
div = float(num1) / float(num2)
# Display the sum
print('The sum of {0} and {1} is {2}'.format(num1, num2, sum))
# Display the subtraction
print('The subtraction of {0} and {1} is {2}'.format(num1, num2, min))
# Display the multiplication
print('The multiplication of {0} and {1} is {2}'.format(num1, num2, mul))
# Display the division
print('The division of {0} and {1} is {2}'.format(num1, num2, div))
```

Conditional Statement

Conditional Statement in Python perform different computations or actions depending on whether a specific Boolean constraint evaluates to true or false. Conditional statements are handled by IF statements in Python.

Major Types of Conditional Statement in Python

1. IF
2. IF...ELSE
3. Nested IF

IF Statement :

IF statement is used to execute or check only one statement. If that block is true then it execute the ode otherwise terminated from the block .

If (condition) :

Statements...

.....

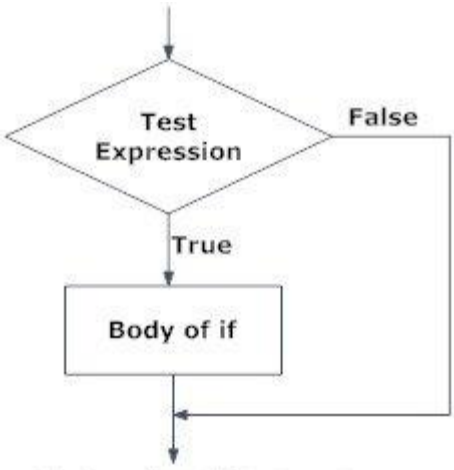

Fig: Operation of if statement

IF Else :

It contains a body of code which runs only when the condition given in the if statement is true. If the condition is false, then the optional else statement runs which contains some code for the else condition.

if expression
 Statement
else
 Statement

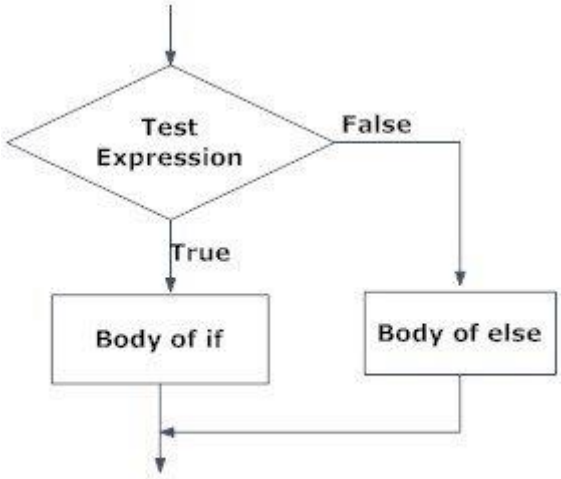

Fig: Operation of if...else statement

Nested IF :

We can have a `if...elif...else` statement inside another `if...elif...else` statement. This is called nesting in computer programming.

Any number of these statements can be nested inside one another. Indentation is the only way to figure out the level of nesting. They can get confusing, so they must be avoided unless necessary.

If (condition):

    If (condition):

        Statement

Looping in Python

Python programming language provides following types of loops to handle looping requirements. Python provides three ways for executing the loops. While all the ways provide similar basic functionality, they differ in their syntax and condition checking time.

1. **While Loop:**
2. In python, while loop is used to execute a block of statements repeatedly until a given a condition is satisfied. And when the condition becomes false, the line immediately after the loop in program is executed.

Syntax :

while expression:
    statement(s)

3. All the statements indented by the same number of character spaces after a programming construct are considered to be part of a single block of code. Python uses indentation as its method of grouping statements.

For Loop

The for loop in Python is used to iterate over a sequence or other iterable objects. Iterating over a sequence is called traversal.

Syntax of for Loop

for val in sequence:
    Body of for

Here, val is the variable that takes the value of the item inside the sequence on each iteration.

Loop continues until we reach the last item in the sequence. The body of for loop is separated from the rest of the code using indentation.

Main differences between "for" and "while" loop

- Initialization, conditional checking, and increment or decrement is done while iteration in "for" loop executed. while on the other hand, only initialization and condition checking in the syntax can be done.
- For loop is used when we are aware of the number iterations at the time of execution. on the other hand, in "while" loop, we can execute it even if we are not aware of the number of iterations.
- If you forgot to put the conditional statement in for loop, it will reiterate the loop infinite times but if you forgot to put the conditional statement in while loop, it will show an error to you.
- The syntax in the for loop will be executed only when the initialization statement is on the top of the syntax but in case of while loop, it doesn't matter whether initialization statement finds anywhere in the syntax.
- The iteration will be executed on if the body of the loop executes. on the contrary, the iteration statement in while loop can be written anywhere in the syntax.